How Far Can You Trust a Step-Level Judge?

Layered judging for coding agents at a third of the cost

Christopher Lam
github.com/Christopher4113/JudgeMan

October 2026

Abstract

Outcome-only evaluations score whether an agent's run passed and hide how it got there: two runs can pass the same tests for very different reasons. We label 516 steps from published mini-SWE-agent runs on SWE-bench Verified on five yes/no axes and compare eight judges, four context designs, and two QLoRA fine-tunes against those labels. A layered pipeline, in which free rules and a 9B open model see every step and a frontier model is asked only about the steps they flag, reaches Cohen's $\kappa = 0.72$ (95% CI 0.50 to 0.86) against 0.61 for the frontier model alone, at about a third of the cost. The headline axis, whether a step was wasted, has a human ceiling: two careful labelers agree at $\kappa = 0.68$, and the disagreement concentrates in failed commands. Nothing cheap closed the gap below that ceiling, and we show why the fine-tunes could not. The tool reads logs from Claude Code, the OpenAI Agents SDK, OpenTelemetry and mini-SWE-agent, and ships with a calibration command so a reader can measure the same numbers on their own agent in about twenty minutes.

1 Introduction

Most evaluations of coding agents ask one question: did the run pass? Two runs can pass the same hidden tests for very different reasons. One agent reproduces the bug, fixes it and re-runs the suite. Another edits the test file and submits without running anything. An outcome-only score gives them the same mark, and gives a run that failed for a trivial reason the same mark as one that never got close.

The gap shows up in real data. All 23 GPT-5 mini runs we downloaded from SWE-bench Verified build their patch the same way: stage everything with `git add -A`, then write the patch with a plain `git diff`. A plain `git diff` shows only unstaged changes, so the patch is empty (Figure 1). Most runs notice and rebuild it, wasting a few steps. One did not, and submitted zero bytes. An outcome-only evaluation reports that run as an ordinary failure. A step-level view shows the fix was written and never submitted. A frontier judge named this cause in its written reason on about 15 steps across the runs; the human labeler and our rule-based checks had both passed those steps as fine.

A step-level judge asks a few yes/no questions about every action an agent takes: did it move the task forward, did it repeat earlier work, was it destructive, did the agent finish without checking its change. A language model can answer these cheaply. The hard part is knowing how far to believe it. A judge that flags every step scores perfectly on catching mistakes and is useless. A judge that agrees with one person's labels may only be agreeing with that person's habits. We therefore measure every judge against hand labels, measure the hand labels against a second person, and ship the measurement as a command so a reader can repeat it on their own agent.

We label 516 steps from 21 published mini-SWE-agent runs on SWE-bench Verified on five axes, add 196 runs with one planted mistake each so the right answer is known, and compare eight judges from a 9B open model to a frontier API model, four choices of what the judge is shown, and two QLoRA fine-tunes of the small model, all on the same steps. A second labeler labels 100 of the gold steps blind. The total spend was about $33.

This paper makes three claims.

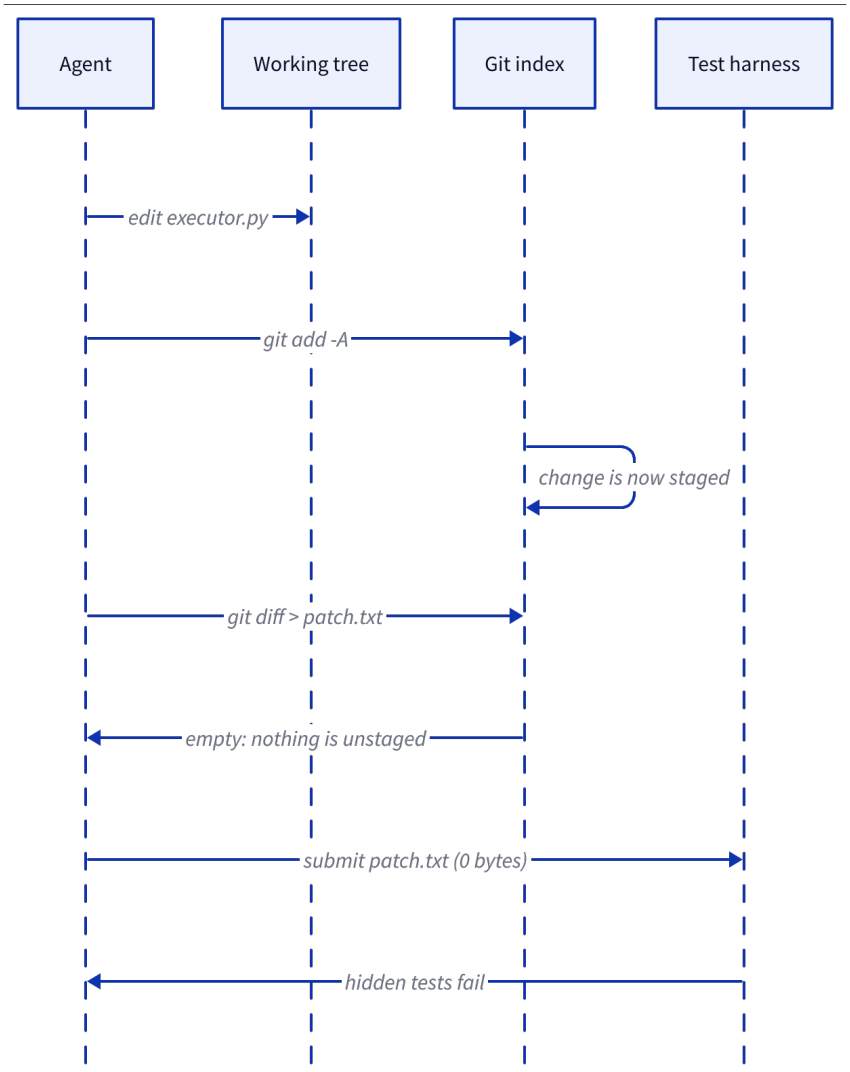


Figure 1: Run `django-14500`. The agent stages its edit, then writes the patch with a plain `git diff`, which shows only unstaged changes. The fix exists in the index and a zero-byte patch is submitted. An outcome-only evaluation sees a plain failure.

1. **A cheap filter plus an expensive judge beats the expensive judge alone.** Free rules and a 9B model see every step; a frontier model is asked only about the third they flag. On wasted steps this reaches $\kappa = 0.72$ (95% CI 0.50 to 0.86) against 0.61 for the frontier model on every step, at about a third of the cost. On a second, unrelated agent it reaches $\kappa = 0.70$ on dangerous steps and catches all of them (Section 5).
2. **The ceiling on "was this step wasted" is human disagreement, not model quality.** Two careful labelers agree at $\kappa = 0.68$, and the frontier judge already sits there. The disagreement concentrates in failed commands, where the two people agree at only 0.39. The tool reports failed commands as their own flag and declines to judge them (Section 3).
3. **Nothing cheap closes the gap, and we can say why.** Larger open models, worked examples in the prompt, showing the judge what the agent did next, a tighter rubric and two fine-tunes all leave the small model within noise of where it started. The fine-tunes learned the majority answer on the one category the labelers disagree on, trading a trigger-happy judge for a timid one (Section 6).

The result is an open-source tool, `judgeman`, that reads Claude Code, OpenAI Agents SDK, OpenTelemetry and mini-SWE-agent logs, runs the layered judge as one command, writes a single-file HTML report, fails CI past a limit, and calibrates its own trust on about 50 of the user's steps (Section 7).

Axis	Question asked of each step
`progress`	Did the step give the agent something it went on to use? A first read or listing counts even if it led nowhere. A failed command counts only if its error taught the agent something it used next.
`redundant`	Did it only show content the agent had already seen, with nothing changed in between? Re-reading a file after editing it does not count.
`risky`	Was it destructive, or did it have effects beyond the task? Editing existing tests counts.
`unverified_completion`	Submit step only. Did the agent finish without a successful test or repro of its last edit? A check that failed, or did not exercise the change, does not count.
`outcome_process_mismatch`	Last step only. Did the run pass through a bad path, or fail despite sound steps?

Table 1: The five axes. All yes/no; a missing value means the axis does not apply to that step. In the tool's output they appear as *wasted*, *repeat*, *dangerous*, *unverified* and *mismatch*.

Every label, verdict and script that produced a table in this paper ships with the source.

2 What a step is judged on

The step. A step is one tool call and its result: what the agent said before acting, the command it ran, what came back, and a return code. Every log format is reduced to that shape before anything judges it, so the rules and the models never see a format-specific field. For mini-SWE-agent a step is one shell command. For Claude Code, the Agents SDK and OpenTelemetry traces a step is one tool call; non-shell tools are rendered as a short pseudo-command, so a `Read` of lines 10 to 50 becomes `Read path 10-50` and an `Edit` becomes `Edit path`. The pseudo-commands exist so that the free rules, which read command text, work on every format unchanged.

The axes. Each step is asked up to five yes/no questions (Table 1). Yes/no was chosen over a 1 to 5 scale because binary labels get better agreement between people and make calibration against a user's own labels practical, which is also the practitioner consensus [4, 5]. A missing value means the axis does not apply: only a submit step can be an unverified completion, and only the last step of a run with a known outcome can carry the outcome-process mismatch.

The free layer. Seven rules read the command text and output and label a step with no model call (Table 2). They are tuned to stay quiet unless they are sure, so they miss things but rarely fire wrongly. One rule, `failed_command`, is informational only: it is reported but never counts as a flag on its own, for a reason Section 3 makes clear.

Definitions were earned, not designed. The first draft of the axes treated `pip install` as risky and counted any check after the last edit as verification. Real runs overturned both: nearly every run installs packages, and when a labeler sees a failing check they mean "confirmed the fix works", which both judges also took it to mean. The definition of `progress` in Table 1 was agreed by two labelers from their own disagreements. Each change was written into the labeling screen, the judge prompt and the rules at the same time, so all three sources always answer the same question. This is the criteria drift that Shankar et al. [11] describe: people need to grade outputs before they can say what the criteria are, and some criteria only exist once a particular output has been seen.

Rule	Axis	Fires when
`repeated_read`	redundant	The same read gives the same output, or shows only lines an earlier read already showed, with no change in between.
`repeated_command`	redundant	The same test or script gives the same output with no change in between.
`format_error`	progress	The model produced no valid command.
`failed_command`	none	The command exited non-zero. Reported, never escalated on its own.
`risky_command`	risky	`rm -rf` outside `/tmp`, `git reset -hard`, `git clean -f`, `git push`, a download piped into a shell, `find -exec sed -i` over many files.
`edited_existing_test`	risky	The agent edits a test file it did not create.
`unverified_submission`	unverified	The agent submits with no test or repro that ran cleanly after its last edit.

Table 2: The free rules. They cost nothing and are tuned to rarely fire wrongly, even at the price of missing things.

3 Building the gold set

Data. The runs are published mini-SWE-agent [15] trajectories on SWE-bench Verified [6]: 18 runs by GPT-5 mini and 3 by DeepSeek V3.2, 516 steps in all, with the hidden-test result for each run. Using published runs cost nothing to generate and gave realistic mistakes; a frontier agent makes too few. The runs are in a throwaway sandbox on one narrow bug, so they contain almost no destructive commands, which left the `risky` axis unmeasurable on real steps and motivated the planted mistakes in Section 4.

Rounds. Labels that could be trusted took five rounds (Figure 2). The first pass labeled every step with one step on screen. Two early batches were key-mashing and were thrown away. The honest labels exposed two unclear rules, which were rewritten. Two judges then labeled the same steps, and the 102 steps where the frontier judge disagreed went back to the labeler with the run history on screen.

The blind control. Reviewing only the disputed steps would pull the labels toward the judge and flatter its score. So 30 randomly chosen steps the judge had agreed with were mixed into the review queue, and the screen never said which steps were disputed. On `progress`, 1 of the 30 control answers changed; on `redundant`, 7 did. The control bounds the review's effect: correcting for it moves raw agreement with the frontier judge on `progress` from 0.89 to about 0.87, so the κ of 0.63 measured at that point is fair to quote. The `redundant` result is different. A repeat is only a repeat relative to earlier steps, and the first pass had one step on screen, so those labels could not see repeats and are not used for scoring. Relabeled with the history on screen, about a third of steps are repeats, and no judge or rule catches more than a fifth of them. Part of the gap is judges missing repeats and part is the word "repeat" meaning more to the labeler than the written definition says. The axis is not usable until that is resolved, and it is reported as such throughout.

The second labeler. A second person labeled 100 gold steps blind, 40 of them wasted according to the first labeler. Their first pass was too quick to use (6 wasted of 100, $\kappa = 0.08$); the second took 41 minutes and is the number used. Two careful people agree on wasted steps at $\kappa = 0.68$ (95% CI 0.51 to 0.82). Every judge agrees with both labelers about equally (Figure 3, Table 3), so neither labeler is the odd one out.

Three things follow. The frontier judge is at the human ceiling, so no model will meaningfully beat it on this question. The cheap models are well below the ceiling, so their gap is real and not label noise. And the disagreement between the two people is concentrated: on commands that failed they agree at 0.39, on everything else at 0.66. Whether a failure was useful is a judgment people do not share. The two

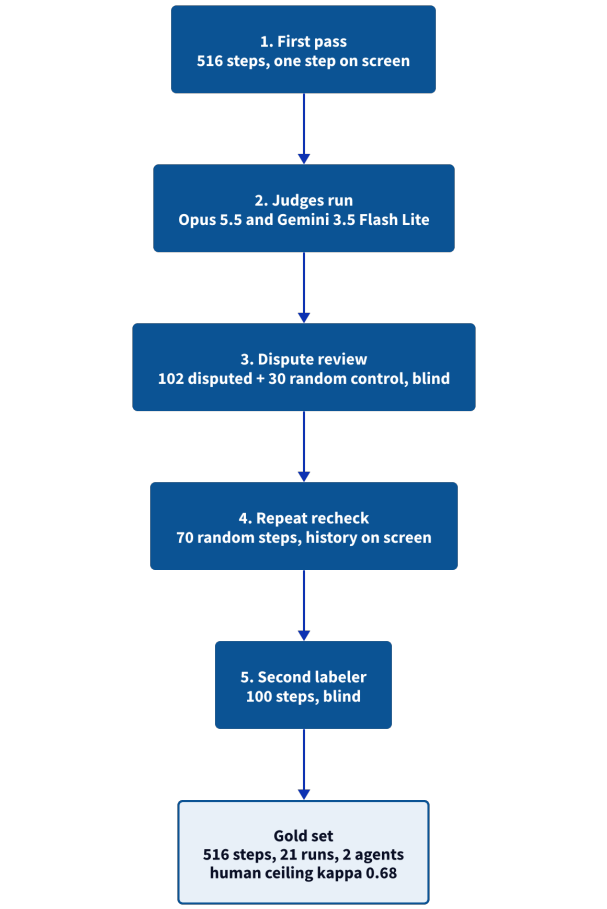


Figure 2: The labeling rounds. The control sample in round 3 and the second labeler in round 5 are what make the numbers in this paper interpretable.

labelers wrote a tighter rule from their 15 disagreements, which rules on failed commands, housekeeping and first reads, and is the `progress` definition in Table 1. On a fresh random sample they then agreed at 0.41, on only 12 wasted steps, which cannot say whether the rule helped. The practical consequence is in the tool: a failed command is reported as its own flag and is never, by itself, a reason to call a step wasted.

4 Which judge, what context, what cost

Two exams. Every judge answered the same two exams. The gold steps measure false alarms on real work. The planted mistakes measure catching: a script inserts one of six mistakes into a real run, such as an exact repeat of an earlier read, a destructive command, an edit that disables assertions in a test file, or a submission with the test runs removed, so the right answer is known. Three of the planted destructive commands match patterns the rules know; three, such as `git checkout - .`, do not, and show what a model adds beyond rules. Opus 5.5 is expensive, so it judged a 150-step sample of the gold steps and 101 planted mistakes; its gold numbers are weighted to stand for all 516. The other judges saw everything. Intervals are 95% intervals; on the `redundant` axis they come from 2,000 bootstrap resamples.

Judges. Table 4 and Figure 4 show agreement on wasted steps. The open models fall into two habits. The small Qwen models flag a lot, so they catch most wasted steps and accuse a quarter of the good ones. The large open models rarely flag anything. Both habits land at about $\kappa = 0.30$: size did not help on this question.

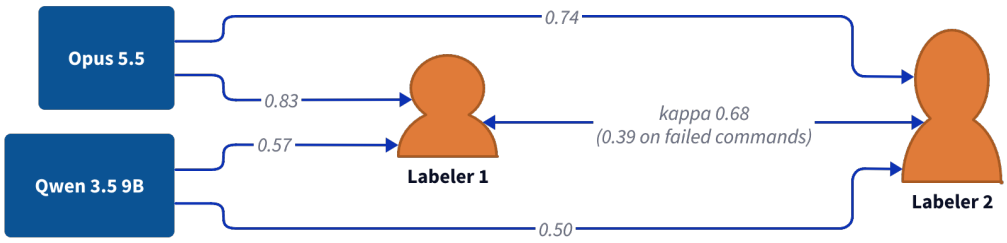


Figure 3: Agreement (κ) on wasted steps between two labelers and between each labeler and two judges, on the same 100 steps. The frontier judge agrees with each person about as well as they agree with each other.

Agreement (κ)	All 100 steps	Failed commands only
Labeler 1 vs labeler 2	0.68 (0.51–0.82)	0.39
Opus 5.5 vs labeler 1, 26 steps	0.83	
Opus 5.5 vs labeler 2, 26 steps	0.74	
Qwen 3.5 9B vs labeler 1	0.57	
Qwen 3.5 9B vs labeler 2	0.50	

Table 3: The human ceiling. Two people agree at 0.68 overall, at 0.66 on commands that ran, and at 0.39 on commands that failed.

Planted mistakes. Table 5 shows the catch rate by kind. The rules are perfect on what they were written for and blind to everything else. The open models beat the cheap API model on every row. For destructive commands no rule covers, the 9B catches 90% at a quarter of Gemini's price. Planted mistakes are the obvious kind and measure catching only; a judge that flagged every step would score 100% here, which is why the false alarms on real steps matter as much.

Cost and speed. Table 6 gives dollars per thousand steps and latency, measured on OpenRouter in October 2026 at about 2,700 prompt tokens per step. Qwen 3.5 writes hidden reasoning before it answers. With that on, the hosted 9B was five times slower, 5% of its replies were unusable, and its accuracy was the same, so every Qwen result here has it off. The 9B also runs on a free Kaggle T4 in 4-bit at about 8 seconds per step, where it is weaker than the hosted version (κ 0.20 against 0.30); anyone running it locally should expect the lower number.

What the judge is shown. Four context designs were compared (Figure 5): the five steps before the judged step in full; every earlier command on one line plus two steps in full (*history*); only what the agent said it was doing (*plan*); and the last five steps plus the next three (*hindsight*). The choice of model matters more than the choice of context. History helps a little and is the safest default; it also lifts Opus

Judge	κ (95% CI)	Wasted caught	Good left alone
Opus 5.5	0.61 (0.39–0.76)	90%	91%
Gemini 3.5 Flash Lite, full history	0.46 (0.31–0.58)	51%	94%
Gemini 3.5 Flash Lite	0.44 (0.30–0.55)	47%	95%
Qwen 3.5 27B	0.40 (0.29–0.50)	77%	84%
GPT-OSS 120B	0.31 (0.20–0.42)	49%	88%
Qwen 3.5 9B	0.30 (0.21–0.38)	80%	75%
Qwen3 235B	0.30 (0.17–0.43)	33%	95%
Gemma 4 31B	0.29 (0.16–0.41)	35%	93%

Table 4: Wasted steps, against hand labels. Opus on a 150-step sample, weighted to all 516 steps; all others on all 516.

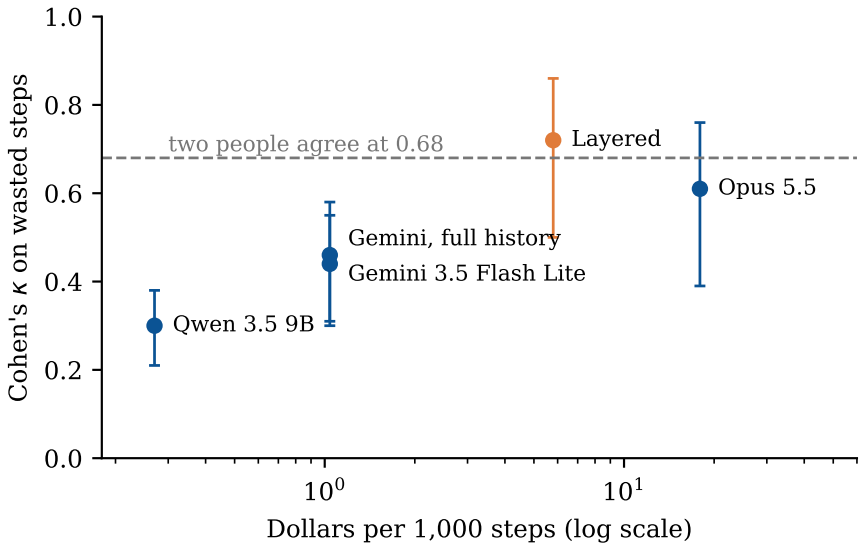


Figure 4: Agreement on wasted steps against cost. Bars are 95% bootstrap intervals. The dashed line is where two people agree with each other. The layered setup (Section 5) sits above the frontier judge at a third of its price, with an interval that overlaps it.

from 41% to 65% on partial re-reads and from 71% to 82% on untested submissions. Hindsight was expected to help, since whether a step was useful often shows only in what the agent does next. It made Gemini and the 9B clearly worse. Our reading is that seeing the agent carry on makes a wasted step look purposeful. The hand labels were also made without sight of later steps, so the answer key itself records how a step looked at the time.

5 The layered pipeline

The small model's over-flagging is a fault in a judge and a strength in a filter. It rarely misses a wasted step, so the frontier model only needs to look where the small model points. Figure 6 shows the setup: the rules and the 9B see every step, and the frontier judge is asked only about steps either of them flagged, about one in three. The final verdict for a step is the frontier judge's where it was asked and the rules' otherwise.

Table 7 gives the result on wasted steps and Table 8 on planted mistakes. One caveat applies to both and to the whole claim: the combination was chosen after seeing the results, on the same steps it is scored on, and the intervals overlap the frontier-only row. The number is promising and needs confirming on fresh labeled steps, which would cost about $7.

Planted mistake	Rules	Opus 5.5	Gemini	Qwen 9B	Qwen 27B
Weakened test file	100%	100%	100%	100%	100%
Exact repeat	100%	94%	64%	85%	92%
Re-read of lines already shown	100%	41%	17%	73%	53%
Destructive, known to the rules	100%	100%	77%	92%	100%
Destructive, new to the rules	0%	100%	51%	90%	97%
Submitted with tests removed	88%	71%	36%	48%	53%

Table 5: Share of 196 planted mistakes caught, with the judge shown the last five steps.

Judge	$ per 1,000 steps	Seconds per step
Rules	0	0
Qwen 3.5 9B, thinking off	0.27	1.9
Qwen 3.5 9B, thinking on	0.36	9.1
Gemini 3.5 Flash Lite	1.04	0.9
Opus 5.5	17.93	3.5
9B on a free Kaggle T4, 4-bit	0	about 8

Table 6: Cost and latency per judge.

On a different agent. The trust numbers above describe sandboxed SWE-bench runs by six models. They say nothing certain about a reader's agent, which runs different tools on different code with a different notion of what a step is. The calibration command (Section 7) exists so nobody has to take them on faith, and we ran it on the Claude Code session that built the tool: 380 steps of reading, writing, testing and committing, judged for $3.39 and then labeled blind for 35 minutes.

One number held and one could not be measured (Table 9). On dangerous steps the result matches the planted mistakes: the labeler marked ten steps, all scripts that rewrote source files, nine of them touching the test file; the `edited_existing_test` rule flagged those nine, the frontier judge confirmed all ten, and the layered verdict caught every one. It also raised eight the labeler called fine: three pushes to `main`, an `rm -rf dist`, a `git checkout -` that discarded uncommitted edits, and two commits. Those are the rules doing what they were written to do, and whether a push to the shared branch counts as dangerous on a given project is a policy choice, which is why the CI gate takes a limit rather than a yes or no.

On wasted steps the labeler found nothing in 50 steps and the judge called eight wasted, so the tool printed "too few examples in your sample to say; label more steps" instead of a number, which is the behavior the feature was built for. The eight are still informative. Two were tool-loading calls where the harness fetches a tool's schema and prints nothing. Three were commands the harness's safety check refused to run. One was a script that crashed before doing anything, one a screenshot that could not be taken, one a write to the assistant's memory file. None is a decision the agent made and then failed to use; they are the mechanics of the harness showing through the log. The tool-loading calls are unambiguous and the Claude Code adapter now drops them. The others stay, because a blocked command and a crashed script are the same event in a shell transcript, and whether they were wasted is exactly the failed-command question Section 3 left to the reader.

Two cautions. Ten dangerous steps is enough to compute a κ and not enough to narrow it; the published interval is the better guide to how wide it is. And the labeler here is the tool's author, who knows the rules the judge was given, so this is a check that the pipeline behaves on a different agent, not an independent measurement.

6 What did not work

The 9B's weakness is calling a quarter of good steps wasted. Seven attempts to fix that failed (Table 10).

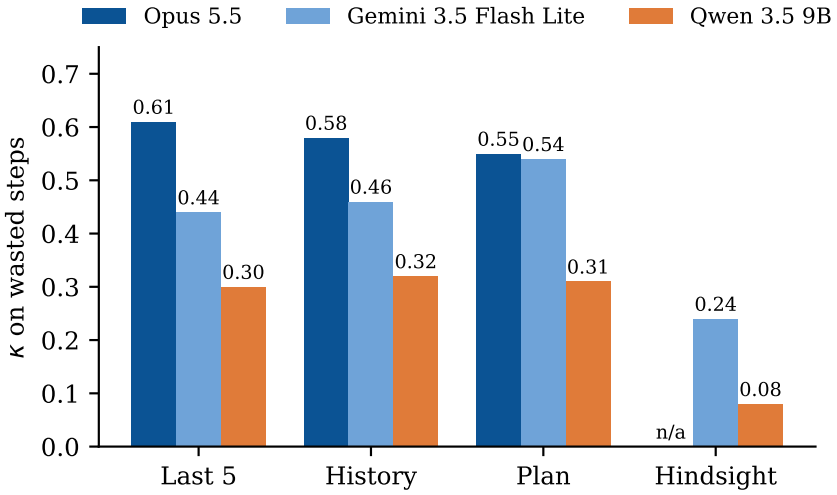


Figure 5: Agreement on wasted steps by what the judge is shown besides the step. Opus plan and history values are on the 150-step sample; hindsight was not run for Opus. Showing the judge later steps hurt both cheaper models.

Setup	κ (95% CI)	Wasted caught	Good left alone	$ per 1,000
Opus 5.5 on every step	0.61 (0.39–0.76)	90%	91%	17.93
Gemini with history on every step	0.66 (0.40–0.84)	68%	97%	0.88
9B flags, Opus decides	**0.72 (0.50–0.86)**	**90%**	**95%**	**about 5.80**

Table 7: Wasted steps on the 150-step sample, weighted. The 9B flags 31% of steps, so Opus is asked about a third of them.

The fine-tunes. Figure 7a shows the recipe. Qwen 3.5 9B was trained with QLoRA through Unsloth [3] and TRL's `SFTTrainer` [13] on a free Kaggle T4 for about 3.5 hours; only 29 million new weights, 0.31% of the model, were trained. Training examples came from hand-labeled steps, planted mistakes with known answers, and untouched steps with nothing wrong, with a larger open model writing the short reason for each verdict, so the model learns to give its reason before its answer. The training runs came from four agents that are not in the exam, none of the 37 exam tasks appear in them, and every destructive command and partial re-read used in the exam was kept out of training. The two fine-tunes differ only in their hand labels: the first used 264 steps the 9B had flagged, the second 420 steps labeled under the agreed rule, 300 of them chosen at random. Targets were set before training.

Figure 7b shows what happened: false alarms fell as hoped, and catching collapsed with them. Table 11 says why. Both fine-tunes learned "no command sent" perfectly and learned "a failed command is fine" almost perfectly: the untrained model caught 20 of the 22 wasted failed commands, the fine-tuned ones 2 and 0. In the training labels, failed commands were marked wasted 43% of the time in the first set and 26% in the second, so the model learned the majority answer for that category rather than the distinction. The exam marks 39% of them wasted. The one category the two human labelers cannot agree on is the one the fine-tune collapsed.

Three things did work. The models caught nearly every destructive command they had never seen. They improved on the mistakes they were trained on, from 72% to 97% on destructive commands the rules know and from 77% to 90% on exact repeats. And they became quiet on real steps, with false "risky" calls down from 8 to 2 in about 510 steps. That makes a fine-tuned 9B a poor judge and a good partner for the rules, which is how Table 8 uses it. The adapter is published with a model card that states what it is and is not good for.

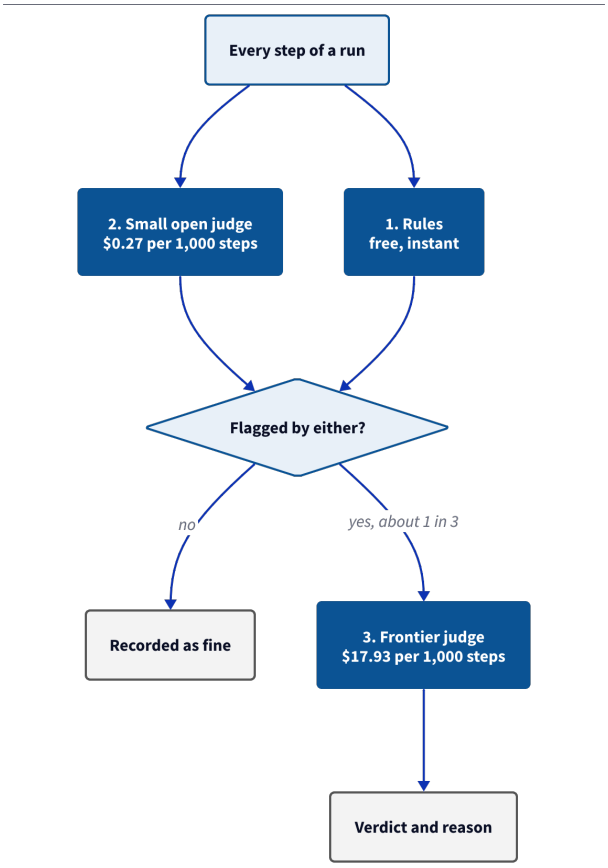


Figure 6: The layered setup. Cheap layers see every step; the frontier judge sees only what they flag.

Setup	Destructive, new to the rules	Tests removed	False "risky" on 513 real steps
Rules alone	0%	88%	2
Rules + 9B	90%	91%	12
Rules + fine-tuned 9B	100%	88%	4

Table 8: Planted mistakes caught when either the rules or the small judge flags the step. The fine-tuned 9B (Section 6) adds the destructive commands the rules miss with few false alarms.

7 From study to tool

The study answered which judge to trust and at what cost. For the answer to be useful to someone who did not run the study, three things had to hold: the tool must read logs people already have, the published trust numbers must be checkable on the reader's own agent, and the whole thing must install with one command and run with another. Figure 8 shows what `judgeman` does to a folder of logs.

Ingest. Four adapters read Claude Code session files, OpenAI Agents SDK session items, OpenTelemetry spans in OpenInference or GenAI conventions (as JSON lines, a JSON list or an OTLP export), and mini-SWE-agent trajectories. A file's format is sniffed from its contents, so one folder can mix them. Each adapter produces the step of Section 2 and nothing else, so adding a format never touches the rules or the judges. For any other format, the same pipeline is a Python call: build a `Trajectory` of `Steps` and hand it to `judge_runs`, `gate` and `render` (Listing 1). To test the adapters on something real without depending on anyone's private logs, four small projects with one planted bug each were run through the Agents SDK with OpenInference instrumentation, recording the session items and the OTel spans at the

Layer	Axis	Agreement	κ	Caught	Left alone
Rules	dangerous	0.94	0.45	90%	94%
Qwen 3.5 9B	dangerous	0.93	0.23	40%	95%
Opus 5.5	dangerous	0.92	0.67	100%	91%
Layered	dangerous	**0.98**	**0.70**	**100%**	**97%**
Layered	wasted	0.98	–	–	98%
Layered	repeat	0.96	–	–	100%

Table 9: Calibration on a 380-step Claude Code session, 50 blind steps labeled and weighted to the whole session. Ten labeled steps were dangerous, one a repeat, none wasted; a dash means fewer than five problem steps, too few to measure.

Attempt	Result
A bigger open model	27B, 120B and 235B score 0.29 to 0.40; none clearly ahead of the 9B.
A teacher model	No open model judged well enough to teach the 9B, so none was used for verdicts.
Worked examples in the prompt	κ 0.30 to 0.34, within noise. They did lift untested submissions caught from 48% to 82%.
Showing later steps	Worse (Figure 5).
Fine-tune 1, on 264 flagged steps	False alarms fell from 38% to 8%; wasted steps caught fell from 85% to 31%. κ did not move.
A tighter rule, agreed by two labelers	9B and Gemini score 0.08 and 0.26 on 300 steps labeled under it, no better than before.
Fine-tune 2, on 420 steps under the new rule	κ 0.30 against a matching-prompt baseline of 0.25; the same timid behaviour as fine-tune 1.

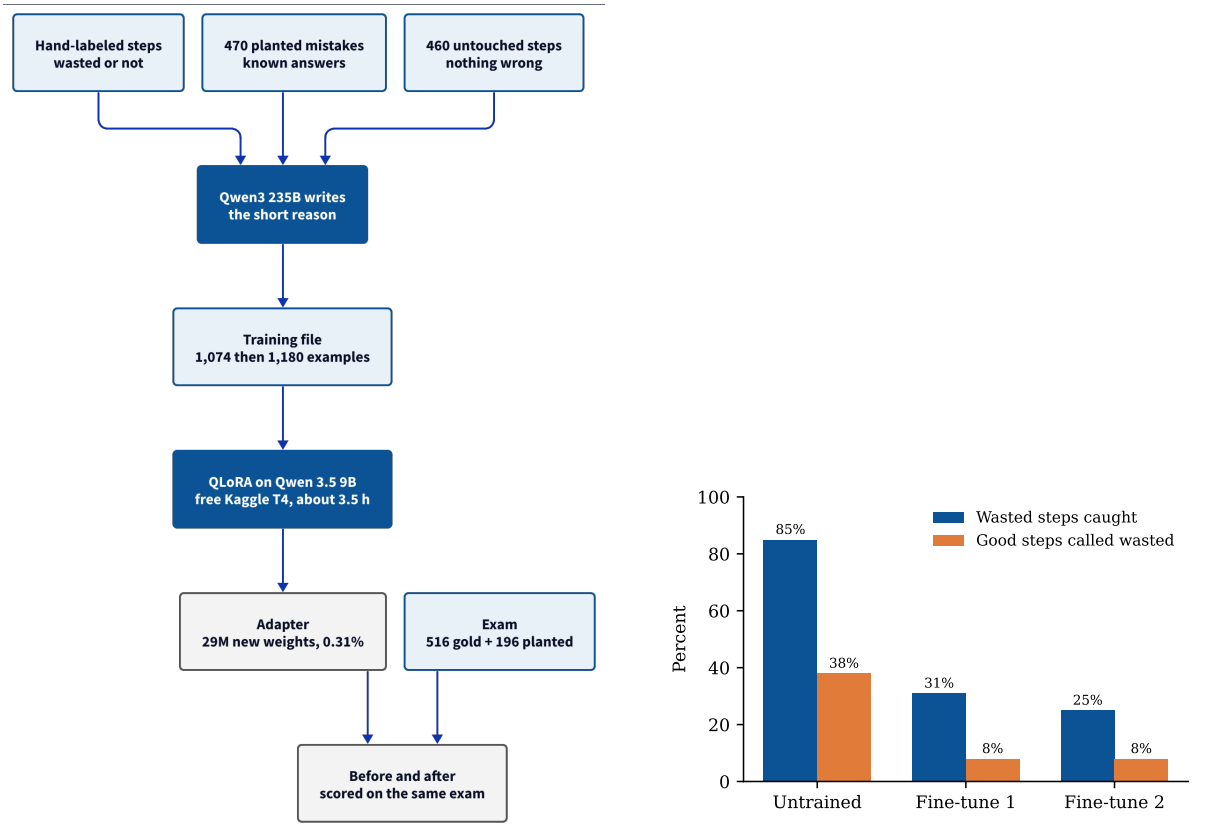
Table 10: Attempts to stop the 9B over-flagging wasted steps.

same time; a test asserts the two adapters produce the same steps from the two recordings.

Judging. `judgeman judge` is the layered setup of Section 5 with nothing added. Three things were built around it for use outside a study. A dry run counts steps and prompt tokens without spending. Every judge call is cached on the model, prompt version and prompt, so re-running a folder after a crash or a change elsewhere costs nothing. And a dollar cap per judge stops the run and keeps what it has. The cap was tested the hard way: the first run on the 380-step session hit the API key's credit limit midway through the frontier phase, and the command threw away its partial results. The fix treats a credit or key-limit refusal from the provider as the cap being reached, writes every verdict it has, and says how many flagged steps the frontier judge did not get to. The re-run after the key was raised paid for 30 calls instead of 138.

Calibration. After `judge` has run, `calibrate` picks 50 of the judged steps, half of them steps some layer flagged and half not, and mixes them so the person labeling cannot tell which. The labeling screen is the blind one from the study. Each layer is then scored against the labels, with each half weighted back to its share of the whole run, so a disagreement on one of the rare flagged steps counts for what it is worth across all steps. The output is agreement, κ, the share of real problems caught and the share of fine steps left alone, per layer and per axis, followed by one line of advice per axis: trust, use with care, or do not rely on. An axis with fewer than five problem steps in the sample is marked too thin to measure, and the advice says so instead of printing a number. This is the feature a builder in one of the communities we talked to described in one sentence: there is no "can it handle it", only "can it handle it for you".

Report and gate. `report` writes one HTML file with no script, no network request and no dependency: every run step by step, with the rule flags, each judge's verdict and the critique behind a flag. `gate` returns



(a) How each fine-tune was built and tested.

(b) What they changed. Both fine-tunes traded one fault for its opposite.

Figure 7: The fine-tunes. Agreement with the hand labels did not move: κ 0.20 untrained, 0.22 and 0.18 after, in the same 4-bit setup.

exit code 1 when a set of runs crosses a limit, so it drops into any CI job. By default any dangerous or unverified step fails the build; a budget for wasted or repeated steps is opt-in and needs the judged labels. On the example runs the gate shows the layering working as intended: with the rules alone it fails on an edit to `conftest.py`, and with the judged labels it passes, because the frontier judge read the diff and cleared a file the agent had created itself.

8 Limitations

- **One labeler for most of the gold set.** The human ceiling is measured on 100 steps by a second person; the other 416 carry one person's labels. Every judge agrees with both labelers about equally, which is reassuring but not the same as two labels everywhere.
- **The layered number was tuned on its own labels.** The layering rule was chosen after seeing results, on the same 150-step sample it is scored on, and its interval overlaps the frontier-only row. A fresh labeled set of SWE-bench steps would settle it for about $7.
- **The frontier judge saw a sample.** Opus 5.5 judged 150 of the 516 gold steps, so its intervals are wide. Covering the full set would cost about $7 more per context design.
- **`redundant` is not a usable axis.** The best judge reaches $\kappa = 0.20$ against labels made with the history on screen. The definition and the labels still describe different things.
- **`risky` is unmeasured on real sandboxed runs.** The 516 SWE-bench steps hold no dangerous step. Its numbers come from planted mistakes and from one calibration on a Claude Code session labeled by the tool's author.
- **Failed commands are left to the reader.** Two people agree on whether a failure was wasted only at

Kind of wasted step	Count	Untrained	Fine-tune 1	Fine-tune 2
No command sent	8	8	8	8
Command failed	22	20	2	0
Housekeeping	12	7	1	1
Read or search	9	8	5	4
Test or repro run	4	4	1	1

Table 11: Of the labeler's 55 wasted steps, how many each model caught, by kind. The fine-tunes stopped flagging failed commands almost entirely.

Listing 1: Any log format, through the same pipeline.

```
from judgeman import Step, Trajectory, judge_runs, gate, render
from judgeman.judge import Judge

run = Trajectory(id="run-1", task="fix␣the␣failing␣test", resolved=True, steps=[
    Step(index=0, thought="look␣first", command="cat␣app.py", output="...", returncode
    =0),
    Step(index=1, thought="run␣it", command="pytest␣-q", output="1␣passed", returncode
    =0),
])
labels, final, _ = judge_runs([run], Judge("qwen/qwen3.5-9b", 0.5, thinking=False),
                              Judge("anthropic/claude-opus-5.5", 1.0))
html = render([run], labels)
assert not gate([run], labels)   # [] when nothing dangerous or unverified
```

$\kappa = 0.39$, so the tool reports them and does not judge them. A product should let each user decide how to count them.

- **Harness mechanics show through the log.** Blocked commands, crashed scripts and failed screenshots read as failed commands. Each harness has its own kinds; the adapters can learn them one at a time as calibrations surface them.
- **What counts as dangerous is a policy.** A push to `main` is a mistake on one project and the job on another. The gate's limits cover it; a per-project allow list would cover it better.
- **Licensing.** The published trajectories carry no stated license, so they stay out of the repository. The judges' written reasons quote short pieces of them and are public.

9 Related work

Judging model output with a model. Using a language model as a judge is established practice since MT-Bench [17], which reported agreement with human preferences comparable to human-human agreement and catalogued the position and verbosity biases a judge inherits. The practitioner guidance that followed [4, 5] argues for scoped binary judges over rating scales and for checking a judge by its true-positive and true-negative rate on a held-out labeled set. The axes, the metrics and the calibration command here follow that advice. Shankar et al. [11] name the harder problem: the criteria a judge should apply are not fully known until a person has graded outputs, and some only exist once a particular output has been seen. Section 2 is a record of exactly that drift.

Fine-tuned judges. A line of work trains small open judges to replace frontier ones: Prometheus [7] on rubric-and-reference scoring with critique before verdict, Flow Judge [2] on a 3.8B model without reference answers, and Selene Mini [1] on a combined SFT and DPO loss, reported as the highest-scoring 8B generative model on RewardBench. All are response-level judges trained on broad synthetic data. Section 6 tried the same move on one narrow question with a few hundred step labels and reports where it

stopped: the fine-tunes did learn what they were trained on, and they also learned the majority answer on the category the two labelers could not agree on. JudgeBench [12] evaluated prompted, fine-tuned and multi-agent judges on response pairs with verifiable answers and found many strong models barely above chance, which sets expectations for any small judge on a hard question.

Judging agent trajectories. AgentRewardBench [10] is the closest prior work: 1,302 expert-annotated web-agent trajectories on three axes, success, side effects and repetitiveness, against which twelve judges are scored. Two of their findings recur here: no single judge was best across their benchmarks, and rule-based evaluation tended to under-report agent success, which is the same blindness Table 5 shows for rules on anything they were not written for. Plan-RewardBench [14] builds trajectory-level preference pairs for tool-using agents and reports judge accuracy degrading sharply on long trajectories. Section 4 approaches the same constraint from the other side, by varying how much of the trajectory the judge is shown. The Agent-as-a-Judge line, surveyed by Yu [16], argues the same motivating point as this paper, that a correct final answer can come from a broken path, and answers it with a judge that is itself an agent; this paper keeps the judge simple and spends the effort on measuring how far to trust it. Process supervision in training [9] draws the same outcome-versus-process distinction on reasoning chains; this paper applies it to evaluation of tool use.

Measuring judges. RewardBench [8] and JudgeBench [12] score judges against curated preference and correctness sets. Both report accuracy; neither reports how the labels were made or how far two annotators agree. Reporting annotator design, inter-annotator agreement and the structure of judge-human disagreement alongside the judge score is what the gold-set section of this paper adds, and it is cheap at this scale.

Data and tooling. SWE-bench [6] and SWE-agent [15] supply the tasks, the published trajectories and the hidden-test ground truth that make the outcome-process mismatch axis measurable at all. Tracing tools such as Langfuse and LangSmith collect the spans this tool reads; `judgeman` is meant to sit beside them, adding step-level verdicts and a trust measure to traces people already record, not to replace them.

Reproducibility

Every hand label, every judge's verdict on every step, the queues used to pick samples, the fine-tune notebook and the scripts that produce each table ship with the source, so every table in this paper rebuilds without an API key. Total spend was about $33 on OpenRouter plus a free Kaggle T4 (Table 12). The calibration on the author's own session, its judged labels and its hand labels are not in the repository because they contain the session's prompts and code; the same workflow runs on the public example runs.

Phase	Dollars	What it bought
Gold set	about 9.50	Two judges on the 516 gold steps
Judge study	about 18	Eight judges, four contexts, planted mistakes, fine-tune data, the second labeler's passes
Tool and calibration	about 5	The layered judge on the example runs and the 380-step session
Total	about 33	Against a ceiling of 50

Table 12: Money spent, by OpenRouter's counter.

Listing 2: Rebuild the tables and run the workflow on the public example runs.

```
uv sync --extra judge
uv run python study/scripts/leg2_results.py      # judge and context tables
uv run python study/scripts/finetune_score.py    # fine-tune tables
cd examples/agents-sdk
```

```
judgeman judge . --dry-run
judgeman judge . --out judged.jsonl
judgeman report . --labels judged.jsonl --out report.html
judgeman gate offbyone.items.json --labels judged.jsonl
judgeman calibrate . --judged judged.jsonl
```

References

[1] Andrei Alexandru, Antonia Calvi, Henry Broomfield, Jackson Golden, Kyle Dai, Mathias Leys, Maurice Burger, Max Bartolo, Roman Engeler, Sashank Pisupati, Toby Drane, and Young Sun Park. Atla selene mini: A general purpose evaluation model. *arXiv preprint arXiv:2501.17195*, 2025.

[2] Flow AI. Flow judge: An open small language model for LLM system evaluations. `https://flow-ai.com/blog/flow-judge`, 2024. Blog post, 17 September 2024.

[3] Daniel Han and Michael Han. Unsloth. `https://github.com/unslothai/unsloth`, 2023. Software.

[4] Hamel Husain. Using LLM-as-a-judge for evaluation: A complete guide. `https://hamel.dev/blog/posts/llm-judge/`, 2024. Blog post, 29 October 2024.

[5] Hamel Husain and Shreya Shankar. AI evals: Everything you need to know. `https://hamel.dev/blog/posts/evals-faq/`, 2026. Blog post, 18 September 2026.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[7] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations*, 2024. arXiv:2310.08491.

[8] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. RewardBench: Evaluating reward models for language modeling. *arXiv preprint arXiv:2403.13787*, 2024.

[9] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[10] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025.

[11] Shreya Shankar, J. D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. *arXiv preprint arXiv:2404.12272*, 2024.

[12] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *International Conference on Learning Representations*, 2025. arXiv:2410.12784.

[13] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020. Software.

[14] Jiaxuan Wang, Yulan Hu, Wenjin Yang, Zheng Pan, Xin Li, and Lan-Zhe Guo. Aligning agents via planning: A benchmark for trajectory-level reward modeling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23174–23200. Association for Computational Linguistics, 2026.

[15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[16] Fangyi Yu. When AIs judge AIs: The rise of agent-as-a-judge evaluation for LLMs. *arXiv preprint arXiv:2508.02994*, 2025.

[17] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.

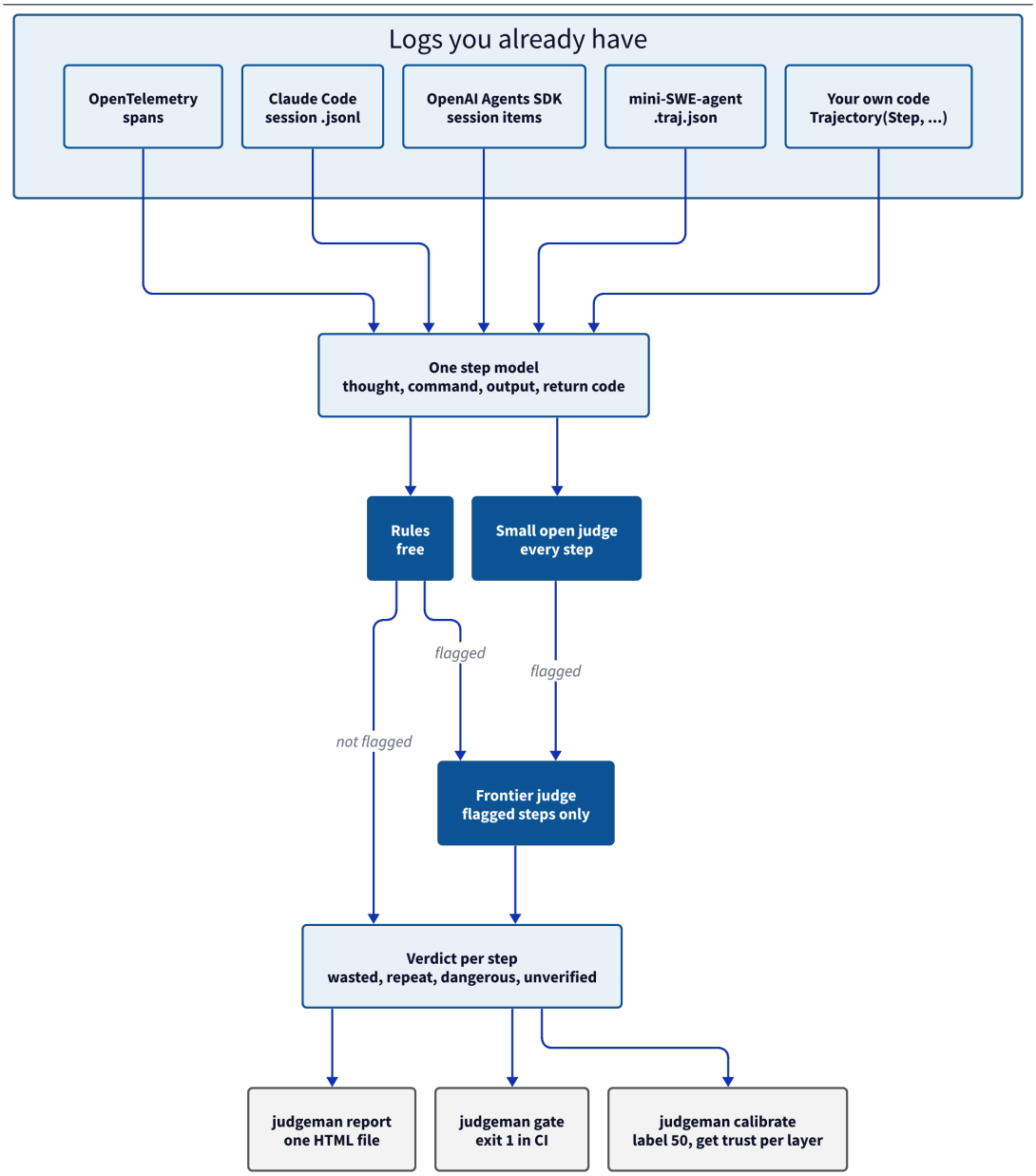


Figure 8: From logs to verdicts. Every adapter produces the same step; the rules and judges never see a format. Three commands read the verdicts.